

Practice Lab:

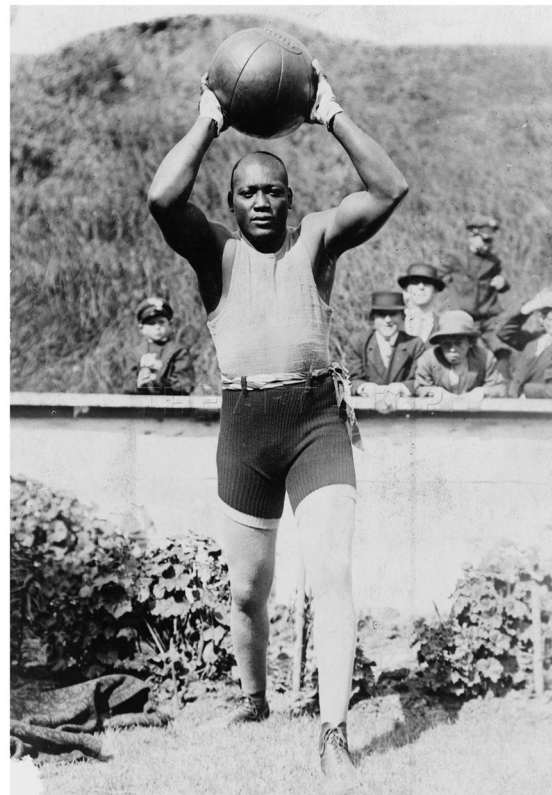
DSE Analytics,
DataFrames

- This Practice Lab is dependent on Discussion Unit 7578, where most of the objects we create in this lab were introduced.
- In this Practice Lab, we use CassandraSQLContext to read from (customer, orders, items), and populate a DSE Core table containing attributes from all 3, including an aggregate.
- Witness predicate pushdown
- Witness optimization

Challenge 1: Prerequisites

Prerequisites:

- Instructions are provided for Linux
- We will operate below recommended settings; 1 OS node only, minimum 8 GB RAM, 12 GB RAM preferred
- That you have an operating single node DSE Core cluster with DSE Analytics enabled.
- All work done as 'root'



From: Maury_Atwater
To: DSE_HOTSHOT

Subject: Need this now !!!

Big deal. You previously loaded customer_orders. ☹

Now join the SQL modeled customer, orders, and items,
and load the DSE modeled customer_orders2. We need
order_total, calculated from items (quantity * price).

I also need Jobs that show:

- . Predicate pushdown
- . Optimization

-MA

Working Goal:



Maury Atwater, President
of Atwater's

Challenge 1: Load DSE from 3 (SQL) tables



- In Practice Lab 7545 we loaded (customers, orders, items); largely one CSV file per one SQL modeled table.
- In Practice Lab 7571, we joined (customer and orders) and loaded a merged DSE table titled, customer_orders.
- Now we need to load customer_orders2 from all 3 SQL tables.
- Data from the third table, items, will be calculated; order_total (from items: quantity * price)
- Use CassandraSQLContext
- Use the Spark REPL

Challenge 1: SQL/CQL Assets

```
DROP KEYSPACE IF EXISTS  
ks_7579;
```

```
CREATE KEYSPACE ks_7579 WITH  
REPLICATION =  
    {'class':  
    'SimpleStrategy',  
    'replication_factor': 1};
```

```
USE ks_7579;
```



Challenge 1: SQL/CQL Assets

```
CREATE TABLE customer
```

```
(
```

```
customer_num
```

```
fname
```

```
lname
```

```
company
```

```
address1
```

```
address2
```

```
city
```

```
state
```

```
zipcode
```

```
phone
```

```
);
```

```
INT PRIMARY KEY,
```

```
TEXT,
```

```
TEXT,
```

```
TEXT,
```

```
TEXT,
```

```
TEXT,
```

```
TEXT,
```

```
TEXT,
```

```
TEXT,
```

```
TEXT
```



Challenge 1: SQL/CQL Assets

```
CREATE TABLE orders
```

```
(
```

```
  order_num
```

```
  order_date
```

```
  customer_num
```

```
  ship_instruct
```

```
  backlog
```

```
  po_num
```

```
  ship_date
```

```
  ship_weight
```

```
  ship_charge
```

```
  paid_date
```

```
);
```

```
INT PRIMARY KEY,
```

```
TEXT,
```

```
INT,
```

```
TEXT,
```

```
TEXT,
```

```
TEXT,
```

```
TEXT,
```

```
TEXT,
```

```
TEXT,
```

```
TEXT
```



Challenge 1: SQL/CQL Assets

```
CREATE TABLE items
(
  item_num          INT,
  order_num         INT,
  stock_num         INT,
  manu_code         TEXT,
  quantity          INT,
  total_price       FLOAT,
  PRIMARY KEY ((order_num), item_num)
);
```



Challenge 1: SQL/CQL Assets

```
CREATE TABLE customer_orders2
(
  order_num          INT,
  paid_date          TEXT,
  //
  customer_num       INT,
  state              TEXT,
  order_total        FLOAT,
  PRIMARY KEY((order_num), customer_num)
);
```



All columns need a value copied or derived from the 3 source tables.

Challenge 1: More ..

- After (n) minutes, cheat; there is a partial solution at the end of this unit (a 2 table/incomplete solution)
- Save your work, you'll need it later.



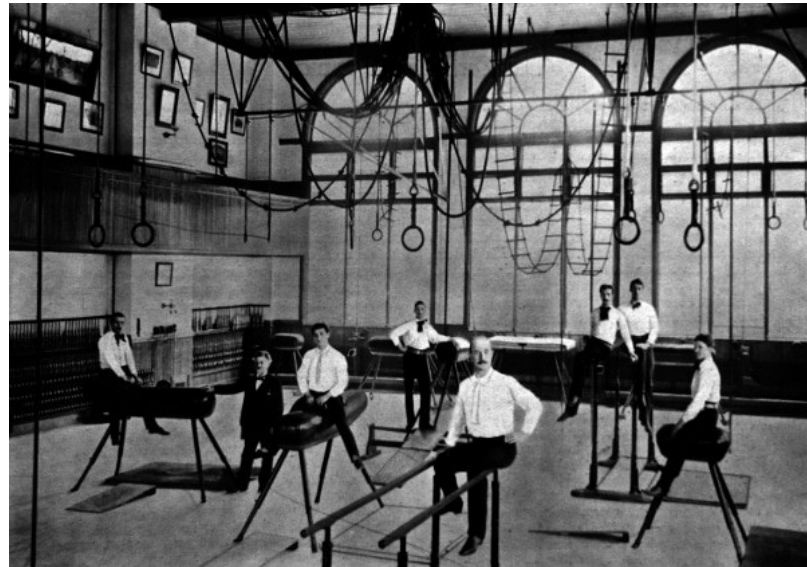
Challenge 2: Predicate Pushdown



- Create the entire runtime environment to replicate the predicate pushdown example from Discussion Unit 7578
- You need/want two DataFrames; Show qualifying and not qualifying.

Challenge 3: DAG Optimization

- Create the entire runtime environment to replicate the DAG Optimization (sort before filter) example from Discussion Unit 7578
- You need/want two DataFrames; One to sort, and a second that filters from the first.



Challenge 4 (Optional): Port Challenge 1 to IDE



- Take the steps you used to complete challenge 1, and move them to a true Scala program.
- Compile and run using the Eclipse/IDE steps outlined in Practice Lab 7554.

Practice Lab:

Lessons Learned





End of Unit:



Additional Detail:

Challenge 1: Hint

```
val recs_DF = spark.  
  sql(  
    "SELECT                                     " +  
    "    t2.order_num,                         " +  
    "    (t3.quantity * t3.total_price) as order_total " +  
    "FROM                                       " +  
    "    ks_7579.orders t2,                   " +  
    "    ks_7579.items t3                     " +  
    "WHERE                                    " +  
    "    t2.order_num =                       " +  
    "    t3.order_num                         " +  
    "GROUP BY                                " +  
    "    t2.order_num,                        " +  
    "    order_total                          " +  
    )
```